ROTEIRO DE APRESENTAÇÃO - ATV 3 COMPUTAÇÃO GRÁFICA

Roteiro Expandido: Transformações Geométricas em CG (8 a 10 minutos)

Dica de ritmo: Fale pausadamente. Quando o roteiro pedir para mostrar o código ou o terminal, realmente pare, aponte para a tela, selecione a linha com o mouse e dê tempo para a turma e o professor lerem.

1. Introdução e o Desafio Técnico (Aprox. 1,5 min)

- **[Ação: Deixe o terminal limpo e o código-fonte aberto nas primeiras linhas]**
- **Fala:** "Boa noite a todos, boa noite professor. Nossa equipe ficou responsável por implementar as transformações geométricas em um **Cilindro fechado, com tampas trianguladas e opposite-face**. O grande desafio deste trabalho foi o requisito de usar o **modo texto**, sem nenhuma biblioteca gráfica pronta, como OpenGL ou Matplotlib."
- **Fala:** "Isso nos obrigou a construir do zero todo o *pipeline* clássico da Computação Gráfica: desde a modelagem matemática dos vértices no espaço 3D, passando pelas multiplicações de matrizes para as transformações, até a projeção matemática para o 2D e a rasterização manual dos pixels — que no nosso caso, são caracteres na tela."

2. Modelagem do Objeto: O Cilindro (Aprox. 1,5 min)

- **[Ação: Mostre a função `create_cylinder` no código]**
- **Fala:** "Antes de transformar o objeto, precisamos construí-lo. Nós definimos o cilindro através de um raio e uma altura, dividindo-o em segmentos. Para cumprir os requisitos específicos do tema:"
    - **Tampas Trianguladas:** "Criamos dois vértices centrais: um no topo e um na base. O algoritmo liga esses centros a cada vértice da borda correspondente, formando fatias triangulares como se fosse uma pizza."

- **Opposite-face (Faces Opostas):** "Para o corpo do cilindro, nós mapeamos os vértices do círculo do topo e traçamos arestas diretas para os vértices correspondentes no círculo da base. Para garantir que a face seja renderizada corretamente, também dividimos essa face retangular lateral em triângulos conectando o vértice do topo ao próximo vértice da base."

- **Fala:** "Todos esses pontos são armazenados usando **coordenadas homogêneas**, ou seja, vetores de 4 posições: $[x, y, z, 1]$. O '1' no final é o que nos permite usar matrizes 4 X 4 para realizar translações através de multiplicações, mantendo o padrão matemático."

3. O Pipeline de Renderização 3D para Texto (Aprox. 1,5 min)

- **[Ação: Mostre as funções `project_3d_to_2d` e `draw_line` no código]**
- **Fala:** "Como não temos uma tela gráfica, precisamos converter nossas coordenadas 3D para uma matriz bidimensional de caracteres que simula nosso SRU (Sistema de Referência do Universo). Fazemos isso em duas etapas:"
  - **1º Projeção:** "Usamos uma projeção isométrica simples com um ângulo de 30 graus para achatar o eixo Z, convertendo (x, y, z) em um par 2D (x, y)."
  - **2º Rasterização:** "Com os pontos em 2D, implementamos o **Algoritmo de Bresenham**. É ele que calcula matematicamente quais 'caracteres' da nossa matriz devem receber o símbolo `#` para traçar uma reta perfeita entre dois vértices, evitando cálculos pesados de ponto flutuante."

4. Exibição do Original e Translação (Aprox. 1,5 min)

- **[Ação: Execute o programa. Role o terminal para cima e mostre APENAS o passo 0 (Original) e o passo 1 (Translação)]**
- **Fala:** "Aqui temos a execução do nosso programa. No passo 0, vocês podem ver a matriz de vértices impressa e o cilindro original renderizado no centro do SRU, marcado pelo símbolo `+`."
- **Fala:** "No passo 1, aplicamos a **Translação**. Na Computação Gráfica, a matriz de translação adiciona fatores de deslocamento na última coluna da matriz de

identidade. Multiplicamos nosso objeto por uma matriz que deslocou $+8$ em X e $+5$ em Y. Como podem ver no desenho, o cilindro se moveu para a direita e para cima no plano. Se olharmos as coordenadas numéricas listadas acima do desenho, os valores exatos refletem essa soma."

5. Escala e Rotação (Aprox. 1,5 min)

- **[Ação: Role o terminal para mostrar os passos 2 (Escala) e 3 (Rotação)]**
- **Fala:** "No passo 2, temos a **Escala**. A matriz de escala altera os valores da diagonal principal da matriz identidade. Nós aplicamos um fator de 0.5 de forma uniforme nos três eixos (x, y, z). O resultado visual é que o cilindro continua na origem, mas seu volume foi reduzido pela metade."
- **Fala:** "Em seguida, no passo 3, aplicamos a **Rotação**. Escolhemos rotacionar 45 graus em torno do eixo Y. A matriz de rotação utiliza as funções trigonométricas de seno e cosseno. Como a rotação é no eixo Y, as coordenadas Y dos vértices permanecem inalteradas, enquanto X e Z são recalculados. Visualmente, nossa malha em texto mostra uma nova perspectiva das diagonais das faces laterais."

6. Cisalhamento, Reflexão e Encerramento (Aprox. 1,5 min)

- **[Ação: Role o terminal para mostrar os passos 4 (Cisalhamento) e 5 (Reflexão)]**
- **Fala:** "No passo 4, demonstramos o **Cisalhamento (Shear)**. Essa transformação causa uma distorção proporcional. Nós aplicamos um cisalhamento no plano XY em função da profundidade Z. Isso significa que quanto mais longe o vértice está no eixo Z, mais ele é deslocado no eixo X. O efeito visual é que o cilindro parece estar 'tombando'."
- **Fala:** "E, por fim, no passo 5, implementamos a bonificação sugerida: a **Reflexão**. Para espelhar o objeto no eixo X, multiplicamos a primeira posição da matriz diagonal por -1. Visualmente o cilindro parece idêntico ao original por ser simétrico, mas se olharmos a lista de coordenadas [aponte para os números], todos os sinais do eixo X foram rigorosamente invertidos."
- **Fala:** "Com isso, validamos na prática toda a teoria vista em sala, aplicando-as desde a modelagem até a renderização manual. Muito obrigado, estamos

abertos a perguntas!"

Como praticar:

Cronometre-se lendo este texto em voz alta enquanto simula as rolagens do terminal. Se ainda assim faltar tempo, você pode gastar mais um minuto explicando **linha por linha de uma das matrizes** (por exemplo, a matriz de rotação que tem os senos e cossenos).